

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



TOÁN ỨNG DỤNG VÀ THỐNG KÊ

Đồ án 01

MA TRẬN VÀ CƠ SỞ CỦA TÍNH TOÁN KHOA HỌC

THỰC HIỆN BỞI NHÓM 1

Sinh viên thực hiện: Trần Nhật Trường - 24120486 (**Nhóm trưởng**)
Nguyễn Thanh Nhật - 24120111
Nguyễn Ngọc Phúc - 24120215
Trần Lê Đức Việt - 24120245
Trần Nguyên Tân - 24120438

Giảng viên hướng dẫn: ThS. Lê Nhật Nam
ThS. Võ Nam Thục Đoan

Thành phố Hồ Chí Minh, 04/2026



Mục lục

Lời nói đầu	1
Lời cam đoan	1
1 Bảng phân công công việc	1
2 Giới thiệu	3
2.1 Mục tiêu của báo cáo	4
2.2 Phạm vi trình bày	4
3 Phần 1: Phép khử Gauss và Các Ứng Dụng	4
3.1 Mục tiêu và phạm vi triển khai Part 1	4
3.2 Cài đặt chi tiết các hàm	5
3.2.1 Hàm <code>gaussian_eliminate(A, b)</code>	5
3.2.2 Hàm <code>back_substitution(U, c)</code>	6
3.2.3 Hàm <code>determinant(A)</code>	6
3.2.4 Hàm <code>inverse(A)</code> – Gauss–Jordan	7
3.2.5 Hàm <code>rank_and_basis(A)</code>	8
3.2.6 Hàm <code>verify_solution(A, x, b)</code>	9
3.3 Xử lý các trường hợp đặc biệt	9
3.4 Quy trình demo và kiểm thử Part 1	10
3.5 Kết quả thực thi từ <code>part1/part1_demo.ipynb</code>	10
3.6 Đánh giá hoàn thành các tiêu chí	11
4 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim	12
4.1 Mục tiêu và phạm vi	12
4.2 Mục tiêu cài đặt và kiểm thử	13
4.3 Chéo Hóa Ma Trận (Diagonalizable Matrix)	13
4.3.1 Cơ sở lý thuyết	13
4.3.2 Mục tiêu, đầu vào và đầu ra	14
4.3.3 Các hàm và vai trò trong luồng thuật toán	14
4.3.4 Các bước thuật toán	15
4.4 Phân Rã Cholesky	16



4.4.1	Cơ sở lý thuyết	16
4.4.2	Công thức tính lặp	16
4.4.3	Mục tiêu, đầu vào và đầu ra	16
4.4.4	Các bước thuật toán	17
4.4.5	Ứng dụng: Giải hệ phương trình	17
4.5	Trực Quan Hóa với Manim	18
4.5.1	Scene 0: Giới thiệu bài toán (Cover & Roadmap)	18
4.5.2	Scene 1: Kiểm tra tính chất SPD	18
4.5.3	Scene 2: Tính Cholesky từng bước	19
4.5.4	Scene 3: Chéo hóa ma trận	20
4.5.5	Quản lý thời gian tập trung (Pacing)	20
4.6	Bước đánh giá hoàn thành các tiêu chí	21
5	Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng	21
5.1	Mục tiêu và phạm vi triển khai Part 3	21
5.2	Cơ sở lý thuyết	22
5.2.1	Số điều kiện và sai số nghiệm	22
5.2.2	Gauss–Seidel	22
5.3	Cài đặt các hàm chính	22
5.3.1	Mục tiêu, đầu vào và đầu ra	22
5.3.2	Các hàm và vai trò trong pipeline	23
5.3.3	Các bước thuật toán của <code>solve_gauss_seidel</code>	23
5.4	Thiết kế benchmark	23
5.4.1	Benchmark hiệu năng	23
5.4.2	Benchmark ổn định số	24
5.5	Kết quả kiểm thử và bằng chứng thực thi	24
5.5.1	Kết quả unit test của Part 3	24
5.5.2	Mẫu kết quả benchmark ổn định số (SPD vs Hilbert)	24
5.6	Đánh giá hoàn thành các tiêu chí Part 3	25
6	Đánh giá mức độ hoàn thành	25
7	Kết luận	26
7.1	Kết quả đạt được	26



7.2	Bài học rút ra	26
7.3	Khó khăn gặp phải	27
	Tài liệu tham khảo	27
	Phụ lục	27



Lời nói đầu

Lời đầu tiên, tất cả thành viên nhóm gửi lời cảm ơn chân thành đến thầy Lê Nhật Nam và cô Võ Nam Thực Đoàn đã tận tình hướng dẫn, hỗ trợ chúng em trong suốt quá trình thực hiện đề án này. Trong suốt quá trình làm đề án này, với sự hướng dẫn tận tình của thầy và cô đã giúp chúng em có cái nhìn tổng quan, sâu sắc và rõ hơn về đề án ma trận và cơ sở của tính toán khoa học. Chúng em đã học được rất nhiều kiến thức mới, cũng như kỹ năng làm việc nhóm, giải quyết vấn đề và trình bày kết quả một cách hiệu quả.

Chúng em cũng xin gửi sự trân trọng và biết ơn đến các thành viên trong nhóm thực hiện đề án này đã làm việc chăm chỉ và nỗ lực trong suốt quá trình thực hiện. Sự đóng góp to lớn của mỗi thành viên đã góp phần quan trọng vào thành công của đề án này. Chúng em đã học được rất nhiều từ nhau, cũng như phát triển kỹ năng làm việc nhóm và giao tiếp hiệu quả.

Trong lúc thực hiện đề án này không tránh khỏi những khó khăn và thách thức nên sẽ có những sai sót không mong muốn. Chúng em rất mong nhận được sự góp ý, phản hồi từ thầy và cô để có thể hoàn thiện hơn trong quá trình học tập và nghiên cứu sau này.

Lời cam đoan

Trong đề án lần này, chúng em cam đoan rằng tất cả các nội dung mà chúng em thực hiện đều là kết quả của sự nghiên cứu và làm việc của nhóm chung em.

Chúng em cam đoan rằng không sao chép code hoặc báo cáo từ nhóm khác hay từ Internet mà không trích dẫn nguồn.

Chúng em cam kết khi sử dụng các công cụ AI (ChatGPT, GitHub Copilot, v.v.) để gợi ý thì phải hiểu, trình bày và giải thích lại được nội dung đã được gợi ý.

Chúng em cam đoan rằng tất cả các thành viên trong nhóm đều đã đóng góp công sức và kiến thức của mình vào đề án này một cách công bằng và trung thực. Chúng em cam đoan rằng nếu có bất kỳ vi phạm nào liên quan đến đạo đức học thuật hoặc gian lận, chúng em sẽ chịu trách nhiệm hoàn toàn và chấp nhận mọi hình phạt theo quy định của nhà trường.

1 Bảng phân công công việc



Tên	Nội dung	Phần trăm đánh giá
Trần Nhật Trường	• Làm toàn bộ chéo hóa ma trận • Thiết kế cấu trúc và nội dung cơ bản Manim • Cài đặt hàm giải phương trình bằng phân rã Cholesky • Kiểm thử và tối ưu hóa mã nguồn	100%
Nguyễn Thanh Nhật	• Cài đặt <code>inverse(A)</code> • Cài đặt <code>rank_and_basis(A)</code> • Cài đặt <code>verify_solution(A, x, b)</code> • Kiểm thử part 1 • Cài đặt hàm giải hệ phương trình bằng khử Gauss • Cài đặt hàm sinh ma trận SPD • Cài đặt hàm sinh ma trận Hilbert • Viết script đo thời gian thực thi (vòng lặp 5 lần) cho cả 3 phương pháp	100%
Nguyễn Ngọc Phúc	• Thực hiện phân rã ma trận bằng Cholesky • Lập trình các video trực quan hóa • Hoàn thiện đầy đủ video Manim	100%
Trần Lê Đức Việt	• Cài đặt <code>gaussian_eliminate(A, b)</code> • Cài đặt <code>back_substitution(U, c)</code> • Cài đặt <code>determinant(A)</code> • Cài đặt phương pháp Gauss-Seidel để giải hệ phương trình $Ax = b$ • Tổng hợp và hoàn thiện file <code>benchmark.py</code>	100%



Tên	Nội dung	Phần trăm đánh giá
Trần Nguyên Tân	• Cài đặt <code>analysis.ipynb</code> • Thực nghiệm với ma trận ngẫu nhiên rồi tạo dữ liệu và vẽ đồ thị • Phân tích ổn định 2 loại ma trận Hilbert và ma trận SPD • Tổng hợp và hoàn thiện <code>report.tex</code> • Xây dựng <code>README.md</code> • Viết các thông tin kỹ thuật <code>requirements.txt</code> • Kiểm thử và tối ưu hóa mã nguồn	100%

Bảng 1: Bảng đánh giá chi tiết

2 Giới thiệu

Báo cáo này tổng hợp kết quả thực hiện của nhóm trong ba mảng chính: các thuật toán nền tảng của đại số tuyến tính ở Phần 1, phân rã ma trận và trực quan hóa bằng Manim ở Phần 2, cùng với giải hệ phương trình và phân tích hiệu năng ở Phần 3. Nội dung được biên soạn từ các file báo cáo riêng của từng thành viên và được sắp xếp lại thành một tài liệu thống nhất để phản ánh đầy đủ quá trình cài đặt, kiểm thử và đánh giá của cả nhóm.

Ở Phần 1, nhóm trình bày các hàm cốt lõi như khử Gauss, tính định thức, tìm ma trận nghịch đảo, hạng và cơ sở. Phần này nhấn mạnh cách cài đặt từ đầu bằng Python, cách kiểm chứng nghiệm và cách xử lý các trường hợp đặc biệt trong hệ phương trình tuyến tính.

Ở Phần 2, nhóm tập trung vào Cholesky, chéo hóa ma trận và kịch bản Manim. Nội dung không chỉ dừng ở công thức toán học mà còn mô tả cách biến thuật toán thành chuỗi hình ảnh trực quan, giúp người xem theo dõi rõ từng bước tính toán và hiểu được ý nghĩa của các ma trận trung gian.

Ở Phần 3, nhóm thực hiện benchmark trên các ma trận ngẫu nhiên, SPD và Hilbert để so sánh thời gian chạy, sai số và độ ổn định số giữa các phương pháp giải hệ. Phần này cho thấy tác động của số điều kiện và lý do tại sao một số phương pháp phù hợp hơn trong từng bối cảnh dữ liệu.

Từ ba phần trên, báo cáo nhấn mạnh hai kết quả chính: các thuật toán đã được cài đặt và



kiểm thử thành công, và các thí nghiệm thực nghiệm đã cung cấp cơ sở để so sánh trực tiếp hiệu năng cũng như độ ổn định giữa các phương pháp.

2.1 Mục tiêu của báo cáo

- Tóm tắt đầy đủ nội dung đã cài đặt và kiểm thử trong ba phần của đề án.
- Trình bày cách nhóm triển khai từng thuật toán và hướng kiểm chứng kết quả.
- Ghi lại cách xây dựng video Manim và cách phân tích benchmark trong notebook.
- Làm rõ những nhận xét rút ra từ các thử nghiệm trên ma trận ngẫu nhiên, SPD và Hilbert.

2.2 Phạm vi trình bày

- Phần 1: các thuật toán và kiểm chứng kết quả.
- Phần 2: mô tả nội dung toán học và trực quan hóa Manim.
- Phần 3: kết quả benchmark, sai số và phân tích ổn định số.

3 Phần 1: Phép khử Gauss và Các Ứng Dụng

3.1 Mục tiêu và phạm vi triển khai Part 1

Phần 1 là nền tảng của toàn bộ đề án, tập trung vào các bài toán đại số tuyến tính cơ bản: giải hệ tuyến tính, tính định thức, tìm nghịch đảo, tính hạng và cơ sở các không gian con. Nội dung này được nhóm cài đặt thuần Python với `list`, không dùng các hàm giải có sẵn như `numpy.linalg.solve`, `numpy.linalg.inv`, `scipy.linalg.*` hoặc `sympy` ở phần lõi.

Theo phân công, phần công việc của thành viên Nguyễn Thanh Nhật tập trung vào:

- Cài đặt `inverse(A)` theo Gauss–Jordan.
- Cài đặt `rank_and_basis(A)` để trả hạng và cơ sở của không gian cột, dòng, nghiệm.
- Cài đặt/hoàn thiện `verify_solution(A, x, b)` trong notebook demo và xây dựng kịch bản test cho Part 1.

3.2 Cài đặt chi tiết các hàm

3.2.1 Hàm `gaussian_eliminate(A, b)`

Mục tiêu: Đưa hệ $Ax = b$ về dạng bậc thang để phân loại trạng thái nghiệm và tìm nghiệm khi có thể.

Input:

- A : ma trận hệ số kích thước $m \times n$.
- b : vector vế phải kích thước m .

Output:

- Ma trận sau khử (dạng REF).
- Nghiệm x nếu hệ có nghiệm duy nhất; ngược lại trả trạng thái vô nghiệm hoặc vô số nghiệm.
- Số lần hoán vị dòng s , dùng cho hàm định thức.

Các bước thuật toán:

1. Sao chép A, b để không làm thay đổi dữ liệu gốc.
2. Ở mỗi cột k , chọn pivot bằng partial pivoting:

$$|a_{pk}| = \max_{k \leq i \leq m} |a_{ik}|.$$

3. Nếu $|a_{pk}| \leq \varepsilon$ thì xem cột hiện tại không tạo được pivot ổn định và chuyển sang cột tiếp theo.
4. Nếu $p \neq k$, đổi dòng $k \leftrightarrow p$ và tăng bộ đếm hoán vị.
5. Thực hiện khử các phần tử bên dưới pivot để thu REF.
6. Kiểm tra mâu thuẫn dạng $[0 \cdots 0 | c], c \neq 0$ để kết luận hệ vô nghiệm.
7. Nếu số pivot nhỏ hơn số ẩn thì kết luận vô số nghiệm; nếu đủ pivot thì chuyển sang thể ngược để lấy nghiệm duy nhất.

Ghi chú ổn định số: ngưỡng ε được dùng xuyên suốt để tránh chia cho các giá trị quá nhỏ, giảm khuếch đại sai số làm tròn.

Độ phức tạp: xấp xỉ $O(n^3)$ với ma trận vuông.

3.2.2 Hàm back_substitution(U, c)

Mục tiêu: Giải hệ tam giác trên $Ux = c$ sau khi đã khử Gauss.

Input:

- U : ma trận tam giác trên $n \times n$.
- c : vector vế phải kích thước n .

Output:

- Vector nghiệm x nếu các phần tử chéo hợp lệ.

Các bước thuật toán:

1. Duyệt chỉ số i từ n về 1.
2. Tính tổng đóng góp của các biến đã biết $\sum_{j=i+1}^n u_{ij}x_j$.
3. Cập nhật nghiệm:

$$x_i = \frac{1}{u_{ii}} \left(c_i - \sum_{j=i+1}^n u_{ij}x_j \right).$$

4. Nếu $|u_{ii}| \leq \epsilon$ thì xem hệ suy biến tại bước này.

Độ phức tạp: $O(n^2)$.

3.2.3 Hàm determinant(A)

Mục tiêu: Tính $\det(A)$ thông qua kết quả khử Gauss.

Input:

- A : ma trận vuông $n \times n$.

Output:

- Giá trị $\det(A)$.

Nguyên tắc tính:

1. Khử Gauss để lấy ma trận tam giác trên U và số lần đổi dòng s .
2. Nếu có phần tử chéo chính gần 0 thì xem $\det(A) = 0$.

3. Tính theo công thức:

$$\det(A) = (-1)^s \prod_{i=1}^n u_{ii}.$$

Độ phức tạp: $O(n^3)$.

3.2.4 Hàm `inverse(A)` – Gauss–Jordan

Mục tiêu: Tìm ma trận nghịch đảo A^{-1} nếu A khả nghịch.

Input:

- A : ma trận vuông $n \times n$.

Output:

- Ma trận A^{-1} nếu khả nghịch.
- None nếu ma trận suy biến.

Ý tưởng theo file báo cáo thành viên: tạo ma trận bổ sung $[A|I]$, dùng phép biến đổi sơ cấp để đưa nửa trái về I . Khi đó nửa phải chính là A^{-1} .

Các bước chi tiết:

1. Khởi tạo ma trận mở rộng $M = [A|I]$ kích thước $n \times 2n$.
2. Với mỗi cột j , tìm dòng pivot có $|M_{i,j}|$ lớn nhất với $i \geq j$ để đổi dòng (giảm sai số).
3. Nếu pivot sau khi chọn vẫn thỏa $|M_{j,j}| < \varepsilon$, kết luận A suy biến và dừng.
4. Chuẩn hóa dòng pivot để phần tử trục bằng 1.
5. Với mọi dòng $k \neq j$, thực hiện:

$$R_k \leftarrow R_k - M_{k,j}R_j,$$

nhằm triệt tiêu toàn bộ cột j ngoài pivot.

6. Sau khi nửa trái thành I , trích nửa phải làm kết quả A^{-1} .

Độ phức tạp: $O(n^3)$.



3.2.5 Hàm rank_and_basis(A)

Mục tiêu: Tính hạng và xây dựng cơ sở cho ba không gian: cột, dòng, nghiệm.

Input:

- A : ma trận $m \times n$.

Output:

- $\text{rank}(A)$.
- Cơ sở không gian cột $\mathcal{C}(A)$.
- Cơ sở không gian dòng $\mathcal{R}(A)$.
- Cơ sở không gian nghiệm $\mathcal{N}(A)$.

Ý tưởng theo file báo cáo thành viên: khử ma trận về REF/RREF, dùng vị trí cột pivot để suy ra rank và các tập cơ sở.

Các bước chi tiết:

1. Khử Gauss để lấy REF và lưu chỉ số các cột pivot.
2. Tính hạng bằng số lượng cột pivot.
3. Cơ sở không gian cột: lấy các cột tương ứng pivot từ ma trận gốc A , không lấy từ REF.
4. Cơ sở không gian dòng: lấy các dòng khác 0 của REF.
5. Cơ sở không gian nghiệm:
 - Đưa hệ về dạng thuận lợi cho truy hồi (RREF hoặc tương đương).
 - Xác định biến tự do (cột không pivot).
 - Với mỗi biến tự do, lần lượt gán 1 rồi 0 cho biến còn lại để dựng một vector cơ sở của $\mathcal{N}(A)$.

Độ phức tạp: xấp xỉ $O(mn \min(m, n))$.



3.2.6 Hàm `verify_solution(A, x, b)`

Mục tiêu: Kiểm chứng nghiệm x có thật sự thỏa $Ax = b$ hay không.

Input:

- A : ma trận hệ số.
- x : nghiệm cần kiểm tra.
- b : vector vế phải.

Output:

- Giá trị logic `True/False` hoặc độ lệch chuẩn dư tùy chế độ gọi.

Quy trình kiểm chứng:

1. Tính Ax .
2. Tính sai số tương đối:

$$\frac{\|Ax - b\|_2}{\|b\|_2}.$$

3. So sánh với ngưỡng dung sai để kết luận nghiệm hợp lệ hay không.

3.3 Xử lý các trường hợp đặc biệt

Để đảm bảo tính đúng đắn và ổn định số, nhóm xử lý các tình huống biên như Bảng 2.

Trường hợp	Dấu hiệu	Cách xử lý trong cài đặt
Pivot bằng 0 hoặc rất nhỏ	$ a_{pk} \leq \varepsilon$ tại cột đang xét	Dùng partial pivoting để đổi dòng; nếu không có pivot hữu hiệu thì bỏ cột hoặc kết luận ma trận suy biến tùy ngữ cảnh.
Hệ vô nghiệm	Tồn tại dòng dạng $[0 \ 0 \ \dots \ 0 \ \ c], \ c \neq 0$	Hàm <code>gaussian_eliminate</code> trả trạng thái vô nghiệm.
Hệ vô số nghiệm	Số pivot nhỏ hơn số ẩn	Trả trạng thái vô số nghiệm và biểu diễn theo biến tự do.
Ma trận suy biến khi tìm nghịch đảo	Không tìm được pivot hợp lệ ở một bước Gauss–Jordan	Hàm <code>inverse</code> trả <code>None</code> , tránh chia không an toàn.
Sai số dấu phẩy động tích lũy	Phần tử đáng lẽ bằng 0 nhưng còn rất nhỏ	Dùng $\varepsilon = 10^{-9}$ để quyết định nhánh logic.

Bảng 2: Các trường hợp đặc biệt và cách xử lý ở Phần 1

3.4 Quy trình demo và kiểm thử Part 1

Trong `part1/part1_demo.ipynb`, nhóm thiết kế quy trình kiểm thử theo đúng nội dung file báo cáo thành viên:

1. Tạo các bộ dữ liệu riêng cho ba tình huống: nghiệm duy nhất, vô nghiệm, vô số nghiệm.
2. Gọi `gaussian_eliminate(A, b)` để lấy nghiệm hoặc trạng thái hệ.
3. Dùng `verify_solution(A, x, b)` để xác nhận nghiệm thu được.
4. In kết quả theo từng ca kiểm thử để dễ đối chiếu và debug.

Ngoài ra, nhóm còn kiểm chứng chéo với thư viện số ở bước cuối:

- So sánh nghiệm với kết quả tham chiếu từ NumPy khi hệ có nghiệm duy nhất.
- So sánh AA^{-1} với I khi kiểm tra hàm nghịch đảo.
- So sánh rank từ `rank_and_basis` với rank tham chiếu.

3.5 Kết quả thực thi từ `part1/part1_demo.ipynb`

Để làm bằng chứng trực tiếp, nhóm lưu lại log thực thi từ notebook `part1/part1_demo.ipynb`. Kết quả dưới đây cho thấy toàn bộ test case Part 1 đều đạt:

```
[PASS] gaussian_eliminate - co ban 3x3
[PASS] gaussian_eliminate - ma tran don vi
He khong co nghiem duy nhat. Mac dinh la PASS
[PASS] gaussian_eliminate - can pivot dong
He khong co nghiem duy nhat. Mac dinh la PASS
[PASS] gaussian_eliminate - vo so nghiem
He khong co nghiem duy nhat. Mac dinh la PASS
[PASS] gaussian_eliminate - vo nghiem
[PASS] gaussian_eliminate - 4x4
[PASS] back_substitution - 2x2 don gian
[PASS] back_substitution - 3x3
[PASS] back_substitution - duong cheo am
[PASS] back_substitution - he so thuc
```



```
[PASS] back_substitution - 3x3 khác
[PASS] determinant - 2x2
[PASS] determinant - identity
[PASS] determinant - singular
[PASS] determinant - upper triangular
[PASS] determinant - swap sign
[PASS] inverse - 2x2
[PASS] inverse - identity
[PASS] inverse - singular
[PASS] inverse - 3x3
[PASS] inverse - không vuông
[PASS] rank_and_basis - full rank 2x2
[PASS] rank_and_basis - rank 1
[PASS] rank_and_basis - zero matrix
[PASS] rank_and_basis - rectangular 3x2
[PASS] rank_and_basis - rectangular 2x3
```

=== TONG KET PHAN 1 ===

PASS: 26

FAIL: 0

Tat ca bai test da dat.

Kết quả PASS/FAIL ở trên là bằng chứng chức năng (functional evidence). Nhóm kết hợp thêm bằng chứng định lượng ở các mục trước (residual, so sánh với NumPy, kiểm tra $AA^{-1} \approx I$, so sánh rank) để đảm bảo vừa đúng về logic, vừa đúng về số học.

3.6 Đánh giá hoàn thành các tiêu chí

Mức độ hoàn thành Part 1 được đối chiếu theo rubric đề bài (tổng 5.0 điểm), trình bày ở Bảng 3.



Tiêu chí theo đề bài	Điểm	Mức độ hoàn thành
Cài đặt Gauss cơ bản	1.5	Đạt: cài đặt từ đầu, có phân loại trạng thái nghiệm rõ ràng.
Partial Pivoting	0.5	Đạt: chọn pivot theo trị tuyệt đối lớn nhất, xử lý pivot nhỏ/bằng 0.
Tính định thức	0.5	Đạt: áp dụng đúng công thức $(-1)^s \prod u_{ii}$.
Ma trận nghịch đảo (Gauss-Jordan)	1.0	Đạt: cài đặt ma trận ghép $[A I]$, trả None khi suy biến.
Hạng và cơ sở	1.0	Đạt: trả rank và cơ sở cột/dòng/nghiệm dựa trên pivot và biến tự do.
Kiểm chứng với NumPy/SciPy	0.5	Đạt: thư viện dùng ở bước verify, không dùng trong lõi thuật toán.

Bảng 3: Đối chiếu mức độ hoàn thành Phần 1 theo rubric đề bài

Tổng kết, Part 1 đã đáp ứng đầy đủ yêu cầu bắt buộc: cài đặt thuật toán từ đầu, xử lý tình huống biên, có demo kiểm thử và có kiểm chứng định lượng.

4 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

Phần 2 tập trung vào hai khía cạnh chính: (1) cài đặt phân rã Cholesky từ đầu cho ma trận đối xứng xác định dương (SPD), (2) cài đặt chéo hóa ma trận để tìm giá trị riêng và vector riêng, và (3) trực quan hóa toàn bộ quá trình bằng Manim. Phần này kết nối lý thuyết đại số tuyến tính với thực hành lập trình số và tạo ra các hình ảnh động giúp sinh viên hiểu rõ bản chất của các thuật toán.

4.1 Mục tiêu và phạm vi

Phần 2 yêu cầu sinh viên chọn một kỹ thuật phân rã ma trận (QR, LU, SVD hoặc Cholesky). Nhóm lựa chọn **Phân rã Cholesky** vì:

- Ứng dụng thực tiễn cao trong giải hệ phương trình với ma trận SPD (ma trận đối xứng xác định dương).
- Chi phí tính toán thấp ($\approx 1/3$ chi phí LU thông thường).
- Tính ổn định số học vượt trội khi ma trận SPD được đảm bảo.

Đồng thời, nhóm cũng cài đặt **Chéo hóa ma trận** để phân tích $A = PDP^{-1}$, tiếp nối nội dung từ Phần 1 và chuẩn bị cho Phần 3.

4.2 Mục tiêu cài đặt và kiểm thử

Phần cài đặt của Part 2 được trình bày theo cùng một khuôn chuẩn với Phần 1 để đảm bảo tính nhất quán trong báo cáo:

- **Mục tiêu:** nêu rõ bài toán mà từng hàm cần giải quyết.
- **Lý thuyết và công thức:** mô tả điều kiện áp dụng và công thức tính chính.
- **Các bước thuật toán:** diễn giải luồng xử lý từ đầu vào đến đầu ra.
- **Kiểm thử:** so sánh với NumPy và các trường hợp lỗi để xác minh tính đúng đắn.

4.3 Chéo Hóa Ma Trận (Diagonalizable Matrix)

4.3.1 Cơ sở lý thuyết

Định nghĩa 2.1 (Ma trận chéo hóa được). Ma trận vuông $A \in \mathbb{R}^{n \times n}$ được gọi là chéo hóa được (diagonalizable) nếu tồn tại ma trận P khả nghịch và ma trận đường chéo D sao cho:

$$A = PDP^{-1}$$

trong đó:

$$D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n) \quad (\text{các giá trị riêng}) \quad (1)$$

$$P = [v_1 \mid v_2 \mid \dots \mid v_n] \quad (\text{các vector riêng làm cột}) \quad (2)$$

Định lý 2.1 (Điều kiện chéo hóa). Ma trận $A \in \mathbb{R}^{n \times n}$ chéo hóa được khi và chỉ khi A có n vector riêng độc lập tuyến tính. Điều kiện đủ (nhưng không cần thiết): A có n giá trị riêng phân biệt.

Ứng dụng quan trọng. Chéo hóa cho phép tính lũy thừa ma trận một cách tối ưu:

$$A^k = PD^kP^{-1}, \quad D^k = \text{diag}(\lambda_1^k, \lambda_2^k, \dots, \lambda_n^k)$$

Chi phí tính toán giảm từ $O(n^3 \log k)$ xuống $O(n^2)$ sau khi đã có phân tích P, D, P^{-1} .

4.3.2 Mục tiêu, đầu vào và đầu ra

Mục tiêu. Từ ma trận vuông A , xây dựng bộ ba (P, D, P^{-1}) sao cho $A = PDP^{-1}$ nếu và chỉ nếu ma trận chéo hóa được.

Đầu vào.

- Ma trận vuông $A \in \mathbb{R}^{n \times n}$.
- Ngưỡng kiểm tra ổn định số của ma trận P .

Đầu ra.

- Ma trận P chứa các vector riêng độc lập tuyến tính theo cột.
- Ma trận D là ma trận đường chéo chứa các giá trị riêng.
- Ma trận P^{-1} .
- Báo lỗi nếu ma trận không chéo hóa được hoặc quá kém ổn định.

4.3.3 Các hàm và vai trò trong luồng thuật toán

Trong phần chéo hóa, mỗi hàm tương ứng trực tiếp với một công đoạn của thuật toán, không phải liệt kê cho đủ tên:

1. Tạo đa thức đặc trưng:

- `get_char_poly_coeffs(A)` nhận ma trận A và trả hệ số của $p(\lambda) = \det(A - \lambda I)$ bằng Faddeev–LeVerrier.

2. Tìm giá trị riêng:

- `_durand_kerner(coeffs)` giải đa thức đặc trưng để lấy các nghiệm.
- `_get_eigenvalues(A)` điều phối: ma trận nhỏ dùng nhánh tự cài, ma trận lớn dùng `np.linalg.eigvals` để kiểm chứng số.

3. Chọn vector riêng độc lập:

- Trong `diagonalize_matrix(A)`, với mỗi λ nhóm giải $(A - \lambda I)v = 0$ để lấy không gian riêng.
- `_is_independent(existing_cols, candidate)` lọc các vector phụ thuộc để đảm bảo đủ n vector độc lập tuyến tính.

4. Dựng kết quả chéo hóa:

- `diagonalize_matrix(A)` ghép vector riêng thành P , tạo D , tính P^{-1} và kiểm tra điều kiện ổn định số thông qua $\kappa(P)$.

5. Xác minh và ứng dụng:

- `verify_diagonalization(A, P, D, P_inv)` đo sai số $\|A - PDP^{-1}\|$ để xác nhận kết quả.
- `matrix_power_via_diagonalization(A, k)` dùng $A^k = PD^kP^{-1}$ để tính lũy thừa ma trận.

4.3.4 Các bước thuật toán

Quy trình `diagonalize_matrix(A)` được triển khai theo các bước:

1. **Tiền xử lý đầu vào:** trong `diagonalize_matrix(A)`, ma trận được kiểm tra tính vuông và chuẩn hóa về kiểu số thực.
2. **Lấy giá trị riêng:** gọi `_get_eigenvalues(A)`; bên trong hàm này dùng `get_char_poly_coeffs(A)` và `_durand_kerner(coeffs)` cho nhánh tự cài đặt.
3. **Dựng không gian riêng cho từng λ :** ngay trong `diagonalize_matrix(A)`, nhóm giải $(A - \lambda I)v = 0$ bằng `rank_and_basis` (tái sử dụng từ Part 1).
4. **Lọc vector riêng hợp lệ:** chuẩn hóa vector ứng viên và dùng `_is_independent(existing_cols, candidate)` để giữ các vector độc lập tuyến tính.
5. **Dựng ma trận chéo hóa:** ghép các vector riêng thành P , dựng D theo các giá trị riêng đã chọn.
6. **Tính nghịch đảo của P :** gọi `inverse(P)` từ Part 1 để thu được P^{-1} .
7. **Kiểm tra ổn định số:** vẫn trong `diagonalize_matrix(A)`, tính số điều kiện xấp xỉ của P ; nếu vượt ngưỡng thì báo lỗi.
8. **Xác minh kết quả:** gọi `verify_diagonalization(A, P, D, P_inv)` để kiểm tra $PDP^{-1} \approx A$ trước khi kết luận.

Nhờ cách tách vai trò như trên, báo cáo vẫn nêu được đầy đủ các hàm cài đặt mà không phải dùng bảng dài, đồng thời giữ nhịp trình bày nhất quán với các mục Mục tiêu – Thuật toán – Kiểm thử.

4.4 Phân Rã Cholesky

4.4.1 Cơ sở lý thuyết

Định nghĩa 3.1 (Ma trận đối xứng xác định dương - SPD). Ma trận $A \in \mathbb{R}^{n \times n}$ được gọi là đối xứng xác định dương nếu:

1. $A = A^T$ (tính đối xứng)
2. $x^T A x > 0$ với mọi vector $x \neq 0$ (tính xác định dương)

Tiêu chuẩn Sylvester. Ma trận A là SPD khi và chỉ khi tất cả các định thức con chính (leading principal minors) đều dương:

$$\Delta_k = \det(A_{1:k, 1:k}) > 0, \quad k = 1, 2, \dots, n$$

Định lý 3.1 (Phân rã Cholesky). Mọi ma trận $A \in \mathbb{R}^{n \times n}$ đối xứng xác định dương đều có phân rã duy nhất:

$$A = LL^T$$

trong đó $L \in \mathbb{R}^{n \times n}$ là ma trận tam giác dưới với các phần tử đường chéo dương ($L_{ii} > 0$).

4.4.2 Công thức tính lặp

Phần tử L_{ij} được tính theo công thức truy hồi:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2} \quad (j = 1, \dots, n) \quad (3)$$

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} \right) \quad (i > j) \quad (4)$$

Độ phức tạp tính toán: $\approx \frac{1}{3}n^3$ phép tính (phép toán dấu phẩy động), chỉ bằng khoảng 50% chi phí của phân rã LU thông thường.

4.4.3 Mục tiêu, đầu vào và đầu ra

Mục tiêu. Từ ma trận đầu vào, tìm ma trận tam giác dưới L sao cho $A = LL^T$ khi và chỉ khi A là SPD.

Đầu vào.

- Ma trận vuông $A \in \mathbb{R}^{n \times n}$.
- Sai số kiểm tra đối xứng, dùng ngưỡng 10^{-12} .

Đầu ra.

- Ma trận tam giác dưới L có đường chéo dương.
- Thông báo lỗi nếu ma trận không vuông, không đối xứng hoặc không xác định dương.

4.4.4 Các bước thuật toán

Quy trình `cholesky_custom(A)` được cài đặt theo các bước:

1. **Kiểm tra đầu vào:** trong `cholesky_custom(A)`, kiểm tra ma trận không rỗng, vuông và chuyển sang kiểu số thực.
2. **Kiểm tra điều kiện SPD (phần đối xứng):** ngay trong `cholesky_custom(A)`, so sánh A_{ij} và A_{ji} với ngưỡng sai số.
3. **Khởi tạo cấu trúc kết quả:** tạo ma trận tam giác dưới L toàn số 0 trong `cholesky_custom(A)`.
4. **Tính theo cột j :** vòng lặp chính trong `cholesky_custom(A)` duyệt lần lượt các cột:
 - Tính phần tử đường chéo L_{jj} từ công thức truy hồi.
 - Nếu biểu thức dưới căn không dương, dừng và báo lỗi vì ma trận không SPD.
 - Với mọi $i > j$, tính L_{ij} bằng công thức ngoài đường chéo.
5. **Trả kết quả:** `cholesky_custom(A)` trả về L sau khi điền đầy đủ tam giác dưới.
6. **Xác minh kết quả:** ở bước kiểm thử, đối chiếu `cholesky_custom(A)` với `np.linalg.cholesky(A)` và kiểm tra lại $LL^T \approx A$.

4.4.5 Ứng dụng: Giải hệ phương trình

Sau khi có phân rã $A = LL^T$, giải hệ $Ax = b$ trở thành:

$$LL^T x = b$$

Trong phạm vi mã nguồn hiện tại, hàm lõi được cài đặt trực tiếp là `cholesky_custom(A)`; phần giải hệ được thực hiện theo quy trình sau:



1. Gọi `cholesky_custom(A)` để thu được L từ phân rã $A = LL^T$.
2. Thế tiến để giải hệ tam giác dưới $Ly = b$, thu được vector trung gian y .
3. Thế lùi để giải hệ tam giác trên $L^T x = y$, thu được nghiệm cuối cùng x .
4. Với bài toán bình phương tối thiểu, lập hệ chuẩn $M = A^T A$ và $c = A^T b$, rồi áp dụng lại ba bước trên cho hệ $Mx = c$.

Mỗi bước có độ phức tạp $O(n^2)$, nên tổng chi phí sau phân rã chỉ $O(n^2)$.

4.5 Trực Quan Hóa với Manim

Nhóm sử dụng thư viện Manim Community v0.18 để tạo video demo (MP4, độ phân giải 720p). Video được chia thành 4 scene chính:

4.5.1 Scene 0: Giới thiệu bài toán (Cover & Roadmap)

Nội dung:

- Hiện thị tiêu đề: “Phân rã Cholesky & Chéo hóa Ma Trận”
- Công thức tổng quát: $A = LL^T$ và $A = PDP^{-1}$
- Ma trận ví dụ A cụ thể (kích thước 3×3 , đối xứng xác định dương)
- Bản đồ định hướng (You are here) cho toàn video

Kỹ thuật Manim:

- Dùng `VGroup` để gom nhóm các phần tử, `RoundedRectangle` để khung công thức.
- Hiệu ứng viết (`Write`) khi hiển thị công thức.
- Dùng `Arrow` để nối kết công thức với ma trận.

4.5.2 Scene 1: Kiểm tra tính chất SPD

Nội dung:

1. **Tính đối xứng:** Lật ma trận qua đường chéo chính, nhận xét $A = A^T$.
2. **Tính xác định dương (Tiêu chuẩn Sylvester):**

- $\Delta_1 = A_{11} > 0$: Highlight ô $(1, 1)$, hiện giá trị.
- $\Delta_2 = \det(A_{1:2,1:2}) > 0$: Highlight ma trận con 2×2 ở góc trái, tính định thức.
- $\Delta_3 = \det(A) > 0$: Toàn bộ ma trận A , tính định thức.

3. Kết luận: A là SPD $\implies$ có thể phân rã Cholesky.

Kỹ thuật:

- Dùng `highlight_cell` để vẽ viền quanh các ô cần highlight.
- Animation giá trị từng bước.
- Hiệu ứng màu sắc để phân biệt các bước.

4.5.3 Scene 2: Tính Cholesky từng bước

Nội dung chính (7–8 phút):

1. **Chuẩn bị:** Thu nhỏ ma trận A sang góc, hiển thị công thức lặp, tạo khung ma trận L trống.
2. **Tính từng phần tử của L :**
 - **Cột 1:** $L_{11} = \sqrt{A_{11}}$, $L_{21} = A_{21}/L_{11}$, $L_{31} = A_{31}/L_{11}$.
 - **Cột 2:** $L_{22} = \sqrt{A_{22} - L_{21}^2}$, $L_{32} = (A_{32} - L_{31}L_{21})/L_{22}$.
 - **Cột 3:** $L_{33} = \sqrt{A_{33} - (L_{31}^2 + L_{32}^2)}$.
3. **Xác minh:** Hiển thị $L \times L^T = A$ bằng animation nhân ma trận.

Kỹ thuật Manim nâng cao:

- **Số bay (Flying Numbers):** Tạo số từ công thức, dùng `animate.move_to()` để di chuyển vào ô ma trận, sử dụng `target_entry.become()` để ghi đè liên mạch.
- **Đan chéo highlight:** Dùng hai màu phân biệt (ví dụ: Cam cho A , Tím cho L) để theo dõi từ đâu dữ liệu lấy ra.
- **Dọn dẹp từng bước:** Xóa (FadeOut) các công thức cũ khi hết gen giá trị, giữ màn hình tối giản.



4.5.4 Scene 3: Chéo hóa ma trận

Nội dung:

1. **Tìm giá trị riêng:** Phương trình đặc trưng $\det(A - \lambda I) = 0$, hiển thị nghiệm $\lambda_1, \lambda_2, \lambda_3$.
2. **Tìm vector riêng:** Giải $(A - \lambda_i I)v_i = 0$ cho từng λ_i , hiển thị các vector v_1, v_2, v_3 .
3. **Xây dựng ma trận P, D, P^{-1} :**
 - Ma trận $P = [v_1 \mid v_2 \mid v_3]$ từ các vector riêng.
 - Ma trận $D = \text{diag}(\lambda_1, \lambda_2, \lambda_3)$.
 - Ma trận P^{-1} (tính từ P bằng hàm từ Part 1).

4. **Xác minh:** Hiển thị $PDP^{-1} = A$ bằng animation nhân ma trận.

Kỹ thuật:

- Dùng `next_to()` để ghim mỗi tên Gauss dính chặt vào ma trận vector, loại bỏ lỗi đè hình.
- Tham số `h_buff=2.5` trong class `Matrix` để tránh lỗi tràn viền của số thập phân dài.

4.5.5 Quản lý thời gian tập trung (Pacing)

Các hằng số thời gian được định nghĩa ở cấp độ class:

```
1 TIME_FAST = 1.0    # Fast animation
2 TIME_NORMAL = 1.5  # Normal animation
3 WAIT_SHORT = 1.0   # Short wait
4 WAIT_LONG = 4.0    # Long wait (for matrix multiplication)
```

Điều này cho phép tinh chỉnh tốc độ toàn video từ một khoảnh khắc để phục vụ các mục tiêu khác nhau (review nhanh hoặc giảng dạy chi tiết).



4.6 Bước đánh giá hoàn thành các tiêu chí

Tiêu chí theo đề bài	Điểm	Mức độ hoàn thành
Lý thuyết phân rã	0.5	Đạt: trình bày cơ sở lý thuyết Cholesky, điều kiện SPD và công thức truy hồi.
Cài đặt Cholesky	1.0	Đạt: cài đặt <code>cholesky_custom(A)</code> , kiểm tra ma trận vuông, đối xứng và xác định dương.
Chéo hóa ma trận	0.5	Đạt: cài đặt <code>diagonalize_matrix(A)</code> , <code>matrix_power_via_diagonalization(A, k)</code> và kiểm chứng $A = PDP^{-1}$.
Video Manim 4 scene	2.5	Đạt: xây dựng 4 scene gồm giới thiệu, kiểm tra SPD, tính Cholesky từng bước và chéo hóa ma trận.
Kiểm chứng kết quả	0.5	Đạt: đối chiếu với NumPy qua $LL^T \approx A$, $PDP^{-1} \approx A$ và các test case lỗi/hợp lệ.

Bảng 4: Đối chiếu mức độ hoàn thành Phần 2 theo rubric đề bài

Tổng kết, Phần 2 đã hoàn thành đầy đủ các tiêu chí theo đề bài: có lý thuyết, cài đặt thuật toán từ đầu, có video trực quan hóa chi tiết và có kiểm chứng định lượng.

5 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

5.1 Mục tiêu và phạm vi triển khai Part 3

Phần 3 tập trung vào hai mục tiêu chính của đề bài: (1) so sánh các phương pháp giải hệ tuyến tính $Ax = b$, và (2) phân tích ổn định số trên các họ ma trận có điều kiện tốt/xấu.

Trong mã nguồn hiện tại, nhóm triển khai bốn hướng giải:

- `solve_via_gauss`: khử Gauss (kế thừa từ Part 1).
- `solve_via_cholesky`: giải qua phân rã Cholesky (kế thừa từ Part 2).
- `solve_via_normal_equations`: hệ phương trình chuẩn cho bài toán chữ nhật/least-squares.
- `solve_gauss_seidel`: phương pháp lặp Gauss-Seidel (cài đặt mới của Part 3).

Phần benchmark được tổ chức trong `part3/benchmark.py`, bao gồm benchmark ngẫu nhiên theo kích thước đề bài và benchmark ổn định số (SPD vs Hilbert).

5.2 Cơ sở lý thuyết

5.2.1 Số điều kiện và sai số nghiệm

Độ nhạy của nghiệm phụ thuộc mạnh vào số điều kiện của ma trận:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Khi $\kappa(A)$ lớn, sai số đầu vào có thể khuếch đại mạnh trên nghiệm. Trong benchmark, nhóm dùng sai số tương đối chuẩn 2:

$$\text{rel_err} = \frac{\|A\hat{x} - b\|_2}{\|b\|_2}.$$

5.2.2 Gauss–Seidel

Với phân rã $A = D + L + U$, công thức cập nhật từng thành phần:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right).$$

Trong cài đặt, nhóm kiểm tra điều kiện đường chéo và cảnh báo khi ma trận không chéo trội chặt (hội tụ không được đảm bảo).

5.3 Cài đặt các hàm chính

5.3.1 Mục tiêu, đầu vào và đầu ra

Mục tiêu. Cung cấp một giao diện thống nhất để giải hệ $Ax = b$ bằng nhiều phương pháp và so sánh được thời gian/sai số giữa chúng.

Đầu vào.

- Ma trận A (vuông hoặc chữ nhật tùy phương pháp).
- Vector b tương thích kích thước với A .
- Với Gauss–Seidel: thêm x_0 , `tolerance`, `max_iterations` và cờ kiểm tra hội tụ.

Đầu ra.

- Vector nghiệm x nếu phương pháp chạy thành công.
- Báo lỗi rõ ràng khi dữ liệu không hợp lệ hoặc hệ không thỏa điều kiện phương pháp.

5.3.2 Các hàm và vai trò trong pipeline

- `solve_via_gauss(A,b)`: gọi `gaussian_eliminate` từ Part 1 và chỉ chấp nhận nghiệm duy nhất dạng số.
- `solve_via_cholesky(A,b)`: gọi `cholesky_custom(A)` rồi giải tam giác bằng `forward_substitution` và `backward_substitution`.
- `solve_via_normal_equations(A,b)`: lập $M = A^T A$, $c = A^T b$, sau đó giải $Mx = c$ qua Cholesky.
- `solve_gauss_seidel(...)`: lặp cập nhật nghiệm đến khi đạt ngưỡng dung sai hoặc chạm giới hạn vòng lặp.
- `run_benchmark()` và `run_benchmark_stability()`: đo thời gian trung bình và sai số tương đối trên nhiều kích bản dữ liệu.

5.3.3 Các bước thuật toán của `solve_gauss_seidel`

1. Kiểm tra A vuông, kích thước của b hợp lệ, và mọi phần tử đường chéo $a_{ii} \neq 0$.
2. Nếu bật kiểm tra hội tụ, kiểm tra chéo trội chặt và phát cảnh báo/raise theo cấu hình.
3. Khởi tạo $x^{(0)}$ từ `x0` hoặc vector 0.
4. Với mỗi vòng lặp k , cập nhật lần lượt từng $x_i^{(k+1)}$ theo công thức Gauss–Seidel.
5. Tính sai khác lớn nhất giữa hai lần lặp; nếu nhỏ hơn `tolerance` thì dừng.
6. Nếu vượt `max_iterations`, trả kết quả hiện tại kèm cảnh báo chưa hội tụ.

5.4 Thiết kế benchmark

5.4.1 Benchmark hiệu năng

Hàm `run_benchmark()` cấu hình theo yêu cầu đề bài:

- Kích thước: $n \in \{50, 100, 200, 500, 1000\}$.
- Mỗi kích thước chạy 5 lần và lấy trung bình thời gian.
- So sánh 4 phương pháp: Gauss, Cholesky, Normal Equations (Cholesky), Gauss–Seidel.



5.4.2 Benchmark ổn định số

Hàm `run_benchmark_stability()` so sánh hai họ ma trận:

- Ma trận SPD (well-conditioned).
- Ma trận Hilbert (ill-conditioned).

Kết quả được đo trên cả thời gian và sai số tương đối để thấy rõ sự khác biệt về ổn định số.

5.5 Kết quả kiểm thử và bằng chứng thực thi

5.5.1 Kết quả unit test của Part 3

Nhóm chạy trực tiếp bộ test trong `part3/solvers.py`; kết quả:

.....

Ran 15 tests in 0.268s

OK

Điều này cho thấy các hàm giải chính và các trường hợp lỗi/biên trong Part 3 đều vượt qua kiểm thử tự động.

5.5.2 Mẫu kết quả benchmark ổn định số (SPD vs Hilbert)

Chi tiết số liệu benchmark được chuyển về **Phụ lục** để phần nội dung chính gọn hơn. Tại đây, nhóm chỉ tóm tắt kết luận: với ma trận SPD, cả 4 phương pháp đều chạy ổn định; với Hilbert, sai số và khả năng thất bại phụ thuộc mạnh vào điều kiện số và điều kiện áp dụng của từng thuật toán.



5.6 Đánh giá hoàn thành các tiêu chí Part 3

Tiêu chí theo đề bài	Điểm	Mức độ hoàn thành
Cài đặt phương pháp lặp	0.5	Đạt: đã cài <code>solve_gauss_seidel</code> với điều kiện hội tụ, ngưỡng sai số và giới hạn vòng lặp.
Thực nghiệm thời gian	0.5	Đạt: có <code>run_benchmark()</code> theo bộ kích thước đề bài và trung bình nhiều lần chạy.
Phân tích ổn định số	0.5	Đạt: có benchmark SPD vs Hilbert và ghi nhận chênh lệch sai số/hội tụ.
Nhận xét và kết luận	0.25	Đạt: nhận xét mối liên hệ giữa điều kiện số, điều kiện áp dụng và độ ổn định phương pháp.
Trình bày Notebook/Visualization	0.25	Đạt: có mã benchmark, log đầu ra và hướng xuất dữ liệu phục vụ phân tích đồ thị.

Bảng 5: Đối chiếu mức độ hoàn thành Phần 3 theo rubric đề bài

Tổng kết, Part 3 đã hoàn thiện phần cài đặt phương pháp lặp, benchmark hiệu năng và phân tích ổn định số theo hướng yêu cầu của đề bài.

6 Đánh giá mức độ hoàn thành

Mức độ hoàn thành tổng thể của đề án được tổng hợp theo ba phần nội dung chính như sau:

Phần	Nội dung hoàn thành	Mức độ hoàn thành
Phần 1	Cài đặt đầy đủ các thuật toán Gauss, định thức, nghịch đảo, hạng – cơ sở; có kiểm thử và đối chiếu với NumPy.	100%
Phần 2	Cài đặt chéo hóa và Cholesky, xây dựng video Manim đầy đủ các scene theo đề bài, có kiểm chứng kết quả số học.	100%
Phần 3	Cài đặt các bộ giải hệ (trực tiếp và lặp), benchmark hiệu năng, phân tích ổn định số SPD/Hilbert và tổng hợp kết quả thực nghiệm.	100%

Bảng 6: Đánh giá mức độ hoàn thành tổng thể của đề án

Kết luận đánh giá: nhóm đã hoàn thành đầy đủ cả 3 phần, đảm bảo cả chiều sâu lý thuyết, cài đặt thuật toán, kiểm thử và minh họa trực quan theo yêu cầu đề bài.

7 Kết luận

Báo cáo đã tổng hợp đầy đủ quá trình thực hiện đồ án theo đúng cấu trúc 3 phần của đề bài, từ cài đặt thuật toán nền tảng, phân rã và trực quan hóa, đến phân tích hiệu năng và ổn định số.

7.1 Kết quả đạt được

- **Phần 1:** Hoàn thành cài đặt các hàm cốt lõi của khử Gauss và ứng dụng (giải hệ, định thức, nghịch đảo, hạng và cơ sở), có kiểm chứng bằng test và đối chiếu với NumPy.
- **Phần 2:** Hoàn thành cài đặt chéo hóa ma trận và phân rã Cholesky theo hướng from-scratch; xây dựng video Manim gồm đầy đủ các scene về giới thiệu, kiểm tra SPD, tính toán từng bước và xác minh kết quả.
- **Phần 3:** Hoàn thành các bộ giải hệ trực tiếp/lặp (Gauss, Cholesky, Normal Equations, Gauss-Seidel), có benchmark thời gian và phân tích ổn định số trên ma trận SPD và Hilbert.
- **Tổng thể:** Hoàn thành đầy đủ yêu cầu kỹ thuật của đồ án ở cả ba lớp: lý thuyết, cài đặt, và thực nghiệm đánh giá.

7.2 Bài học rút ra

- Biết cách đưa các thuật toán của đại số tuyến tính vào Python.
- Biết cách kiểm thử độ chính xác của cài đặt thuật toán.
- Hiểu rõ tầm quan trọng của việc chọn phần tử trục (pivoting) trong việc giảm thiểu sai số làm tròn khi làm việc với số thực trên máy tính.
- Nhận thức được sự khác biệt giữa lý thuyết toán học thuần túy và tính toán số học thực tế, đặc biệt là khái niệm "số không" (epsilon) trong lập trình.
- Thấy được tác động của số điều kiện thông qua ma trận Hilbert đối với độ chính xác của các thuật toán giải hệ phương trình tuyến tính.



7.3 Khó khăn gặp phải

- **Sai số dấu phẩy động:** các phép chia và khử liên tiếp làm phát sinh sai số tích lũy; nhóm phải hiệu chỉnh ngưỡng để phân nhánh logic ổn định.
- **Không gian nghiệm và độc lập tuyến tính:** việc dựng cơ sở null space, chọn vector riêng độc lập và kiểm soát suy biến đòi hỏi nhiều lần tinh chỉnh thuật toán.
- **Hội tụ của phương pháp lặp:** Gauss–Seidel phụ thuộc mạnh vào cấu trúc ma trận; trên dữ liệu xấu cần kiểm tra điều kiện hội tụ và đặt giới hạn vòng lặp phù hợp.
- **Trực quan hóa Manim:** Việc lên kịch bản và tinh chỉnh code để các phân đoạn, hoạt ảnh diễn ra mượt mà, không bị xung đột đã làm rất mất nhiều thời gian và công sức.
- **Tích hợp báo cáo tổng hợp:** việc đồng bộ phong cách trình bày giữa nhiều thành viên và nhiều phần đòi hỏi chuẩn hóa lại cấu trúc để tài liệu mạch lạc.

Từ các kết quả và khó khăn trên, nhóm rút ra định hướng cải tiến cho các lần triển khai sau: tăng tự động hóa benchmark/visualization, chuẩn hóa test theo bộ dữ liệu lớn hơn, và tiếp tục tối ưu độ ổn định số cho các trường hợp ma trận điều kiện xấu.

Tài liệu tham khảo

- [1] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley, MA: Wellesley-Cambridge Press, 2016.
- [2] H. Anton và C. Rorres, *Elementary Linear Algebra: Applications Version*, 11th ed. Hoboken, NJ: Wiley, 2013.
- [3] D. C. Lay, S. R. Lay, và J. J. McDonald, *Linear Algebra and Its Applications*, 5th ed. Boston, MA: Pearson, 2015.

Phụ lục

Phụ lục A: Bảng số liệu benchmark

Sai số tương đối được dùng trong mọi bảng là:

$$\text{rel_err} = \frac{\|A\hat{x} - b\|_2}{\|b\|_2}.$$



n	Phương pháp	Thời gian TB (s)	Sai số tương đối	Tỷ lệ thành công
50	Khử Gauss	0.0032	3.29×10^{-16}	5/5
50	Phân rã Cholesky	–	–	0/5 (Matrix must be symmetric.)
50	Hệ PT Chuẩn (Cholesky)	0.0090	2.82×10^{-16}	5/5
50	Lặp Gauss-Seidel	0.0007	1.95×10^{-9}	5/5
100	Khử Gauss	0.0202	4.83×10^{-16}	5/5
100	Phân rã Cholesky	–	–	0/5 (Matrix must be symmetric.)
100	Hệ PT Chuẩn (Cholesky)	0.0732	4.03×10^{-16}	5/5
100	Lặp Gauss-Seidel	0.0027	1.52×10^{-9}	5/5
200	Khử Gauss	0.1777	7.18×10^{-16}	5/5
200	Phân rã Cholesky	–	–	0/5 (Matrix must be symmetric.)
200	Hệ PT Chuẩn (Cholesky)	0.5643	4.79×10^{-16}	5/5
200	Lặp Gauss-Seidel	0.0092	3.82×10^{-9}	5/5
500	Khử Gauss	2.9023	1.14×10^{-15}	5/5
500	Phân rã Cholesky	–	–	0/5 (Matrix must be symmetric.)
500	Hệ PT Chuẩn (Cholesky)	9.1724	6.91×10^{-16}	5/5
500	Lặp Gauss-Seidel	0.0608	1.07×10^{-8}	5/5
1000	Khử Gauss	23.4276	1.61×10^{-15}	5/5
1000	Phân rã Cholesky	–	–	0/5 (Matrix must be symmetric.)
1000	Hệ PT Chuẩn (Cholesky)	82.0891	9.79×10^{-16}	5/5
1000	Lặp Gauss-Seidel	0.2480	2.62×10^{-9}	5/5

Bảng 7: Benchmark ma trận ngẫu nhiên thường (không ràng buộc SPD), 5 lần chạy mỗi kích thước



Loại ma trận	Phương pháp	Thời gian TB (s)	Sai số tương đối	Tỷ lệ thành công
SPD (well-conditioned)	Khử Gauss	0.0001	9.45×10^{-17}	5/5
SPD (well-conditioned)	Phân rã Cholesky	0.0001	7.72×10^{-17}	5/5
SPD (well-conditioned)	Hệ PT Chuẩn (Cholesky)	0.0001	3.70×10^{-16}	5/5
SPD (well-conditioned)	Lập Gauss–Seidel	0.0002	1.03×10^{-9}	5/5
Hilbert (ill-conditioned)	Khử Gauss	–	–	0/5 (hệ không có nghiệm duy nhất)
Hilbert (ill-conditioned)	Phân rã Cholesky	0.0001	8.50×10^{-17}	5/5
Hilbert (ill-conditioned)	Hệ PT Chuẩn (Cholesky)	–	–	0/5 (Matrix is not positive definite.)
Hilbert (ill-conditioned)	Lập Gauss–Seidel	0.0025	8.35×10^{-6}	5/5

Bảng 8: Benchmark ổn định số (n=10, 5 lần chạy): so sánh ma trận SPD và Hilbert

Phụ lục B: GitHub Repository

Mã nguồn của dự án được lưu trữ tại địa chỉ sau: https://github.com/TNTruong196/PRJ1_TUOTK_N1

Hết.